

Algoritmos y Estructuras de Datos II



UADE



MEJOR EDUCACIÓN PARA MUCHOS MÁS

Algoritmos y Estructuras de Datos II

Comité de ciencias
de la OTAN

UADE



MEJOR EDUCACIÓN PARA MUCHOS MÁS



Algoritmos y Estructuras de Datos II

Comprender y aplicar el concepto de Tipos de Datos Abstractos (TDA) y las estructuras de datos fundamentales, desarrollando la capacidad de analizar problemas, modelar la organización de los datos y diseñar algoritmos eficientes que permitan construir programas correctos, mantenibles y escalables.

Implica aprender a pensar los programas como la combinación de estructuras de datos y algoritmos, entendiendo cómo se representan los datos, cómo se organizan y cómo se operan sobre ellos para resolver problemas computacionales de manera eficiente.

Algoritmos y Estructuras de Datos II

Brindar
conocimientos
fundamentales

Desarrollar
habilidades
prácticas

Fomentar la
innovación

Enseñar ética y
responsabilidad
profesional

Preparar para el
mercado laboral



Conociendonos...

UADE



www.linkedin.com/in/alfonso-fernandez-buttera-87554b4

Profesor Alfonso Fernández Buttera

Clase
1

Unidad 1: Pensamiento Abstracto y TDA

Clase
2-3

Unidad 2: De la abstracción a la implementación

Clase
4

Unidad 3: Diccionario y expresión en Java

Clase
5-6-7

Unidad 4: Implementaciones dinámicas y eficiencia

Clase
8-9

Unidad 5: Árboles

Clase
10-11

Unidad 6: Árboles avanzados

Clase
12-13

Unidad 7: Grafos

PRIMER PARCIAL



Trabajo
práctico

Evaluaciones



Σ PUNTAJES

Casos
Ejercicios
Tareas
1º Parcial
TPO

TRABAJO
PRACTICO

17-Jun-2026

PRIMER EXAMEN
PARCIAL

22-Abr-2026

FINAL

15-Jul-2026

FINAL ADELANTADO
RECUPERATORIOS

01-Jul-2026

RAZONAMIENTO TÉCNICO

TOMA DE DECISIONES FUNDAMENTADAS
CAPACIDAD DE INTEGRACIÓN CONCEPTUAL



CREATIVIDAD

Algoritmos y Estructuras de Datos II

"La Suerte es lo que sucede cuando la preparación se encuentra con la oportunidad" Seneca

(60DC)

**Pensar antes de programar:
¿qué es un TDA?**

Comprender el concepto de TDA y poder leer/escribir una especificación básica y su "traducción" a Java a nivel interfaz/contrato.

Software parte del Hardware



ALAN TURING

DOMINIO GRANDES EQUIPOS



50'



Lenguaje de máquina



SKETCHPAD

En 1963 de la mano de Shuterland nace el primer programa que permitió la manipulación de objetos gráficos

Programación Estructurada

UADE

"Software Engineering" CONFERENCIA OTAN 1968



Eiffel es un lenguaje de programación puramente orientado a objetos para mejorar la fiabilidad y calidad del desarrollo de software comercial mediante principios de ingeniería rigurosos.

Usamos deliberadamente el término ingeniería de software para provocar el pensamiento, sugiriendo que el desarrollo de software debería basarse en los principios de la ingeniería



ALAN KAY

SMALLTALK-80 THE LANGUAGE AND ITS IMPLEMENTATION

1970 a 1980

Metodologías – Modelo de desarrollo de software



Ole-Johan Dahly Kristen Nygaard



Llevar OO al mundo industrial: C with Classes → C++



1980 a 1990 Internet



SISTEMAS ORIENTADOS A OBJETOS

EVOLUCION DE A&ED



2001



UNIFIED MODELING LANGUAGE



Bases de Datos Orientadas a objetos



Python OOP



POO moderna = menos herencia rígida, más composición + modelos de dominio más claros

"GoF": patrones de diseño y OO como lenguaje común de arquitectura

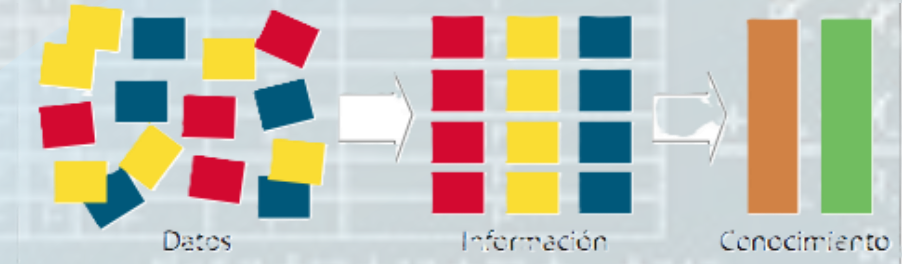
Data-Oriented

OBJETIVO FUE SIEMPRE EL MISMO

Organizar los datos y diseñar algoritmos que permitan construir software cada vez más confiable, eficiente y escalable. Por eso Algoritmos y Estructuras de Datos sigue siendo una de las materias fundamentales de la informática

Un **dato** es simplemente una representación de información que el programa puede almacenar y manipular.

FILOSOFÍA DE DISEÑO



Tipos de Datos



int

Dominio: números enteros

Operaciones: suma, resta, multiplicación, división, comparación.

String

Dominio: cadenas de texto

Operaciones: concatenar, comparar, obtener longitud.

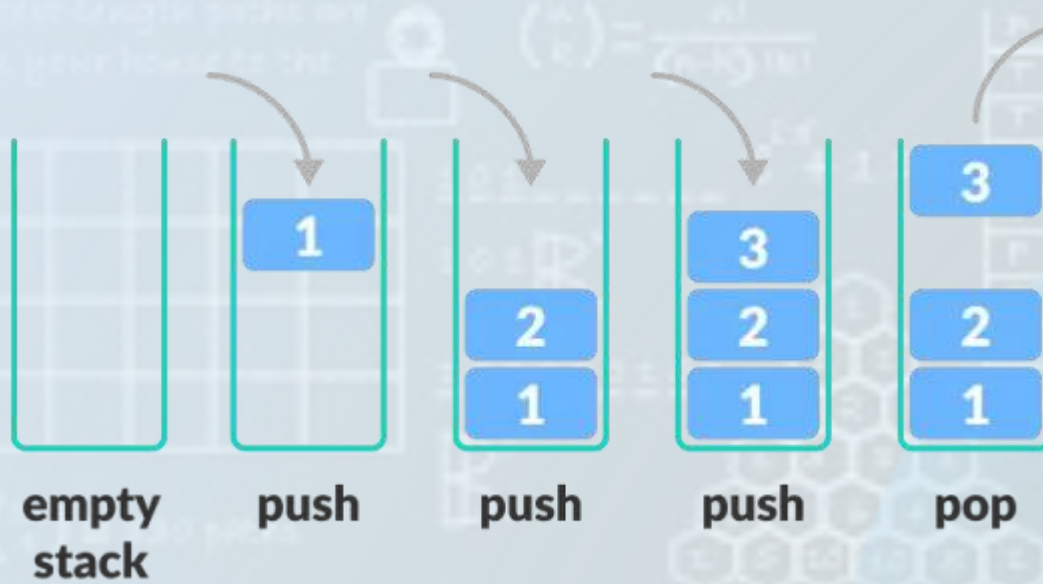
boolean

Dominio: verdadero o falso

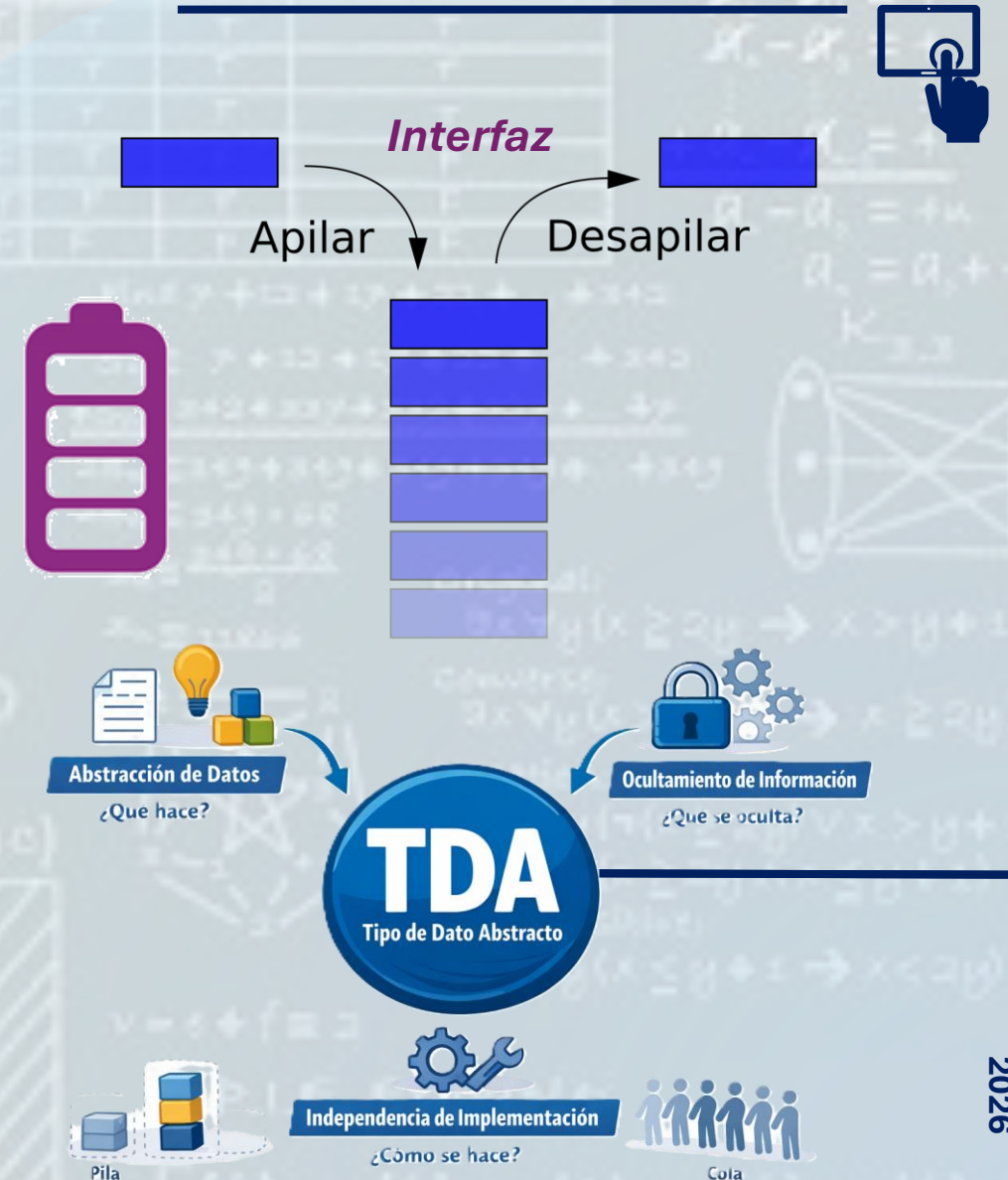
Operaciones: AND, OR, NOT.

Un **Tipo de Dato Abstracto (TDA)** o **Abstract Data Type (ADT)** es un modelo matemático que define un conjunto de datos y las operaciones que se pueden realizar sobre ellos, sin especificar cómo se implementan internamente





FILOSOFÍA DE DISEÑO



Concepto	Rol en el TDA
Abstracción	Define el modelo conceptual simplificado.
Interfaz	Expone las operaciones permitidas al usuario.
Encapsulamiento	Mantiene unidos los datos y sus operaciones.
Ocultamiento	Impide el acceso directo a la representación interna.

Precondición y contrato

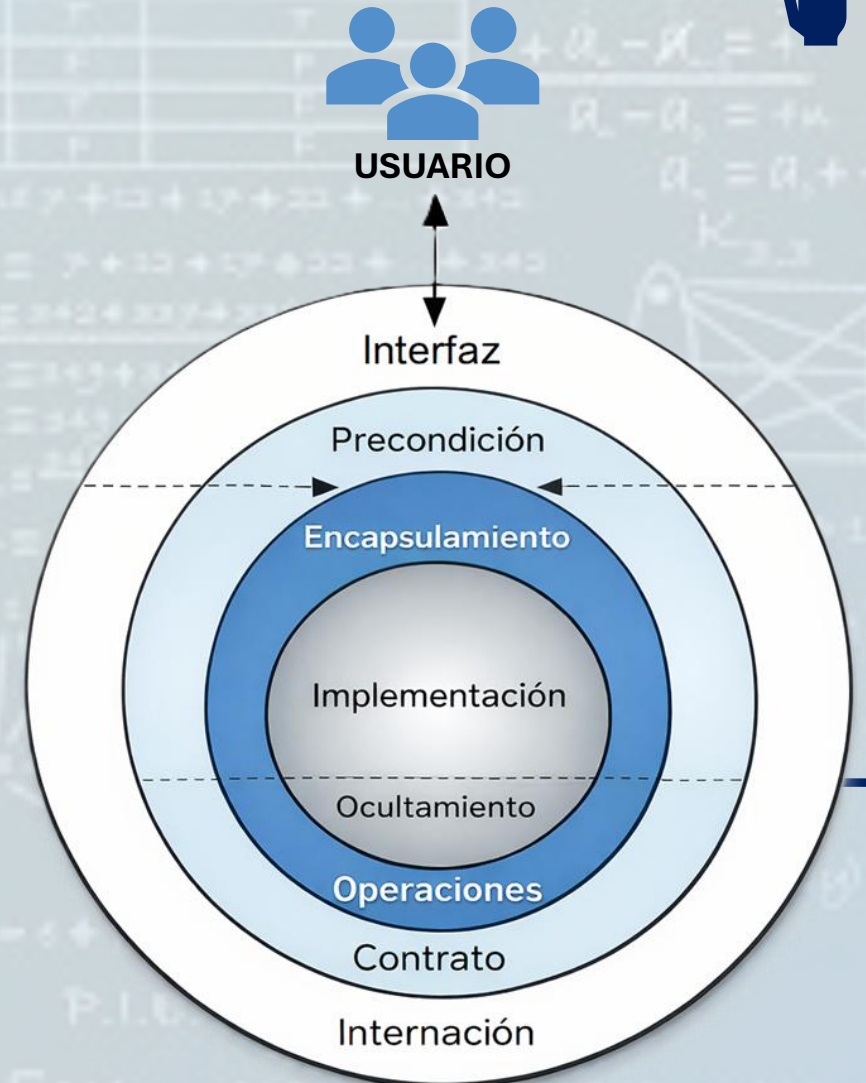


Una **precondición** es una condición que debe cumplirse antes de ejecutar una operación.

Por ejemplo, no tiene sentido quitar un elemento de una pila vacía. Por lo tanto, la operación “desapilar” requiere como precondición que la pila no esté vacía.

Pensar en **contrato** es pensar en términos profesionales. El usuario del TDA se compromete a respetar las precondiciones y por lo tanto La implementación se compromete a garantizar el comportamiento especificado si esas condiciones se cumplen.

Significa que cada módulo del sistema tiene responsabilidades claras.





Notación Básica



1 Nombre del TDA

TDA Pila

2 Lista de operaciones: Se escribe el nombre de la operación y los parámetros.

Apilar(x)

Desapilar()

Tope()

PilaVacía()

3 Tipo de retorno: Si la operación devuelve algo, se indica con una flecha.

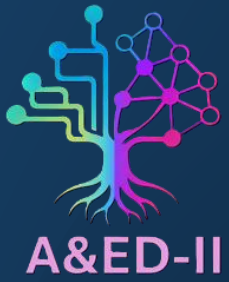
Tope() → elemento

PilaVacía() → boolean

4 Precondiciones: Condiciones que deben cumplirse antes de usar la operación.

Desapilar()

Pre: la pila no está vacía



Actividad



Diseñando un Tipo de Dato Abstracto

*Cada grupo deberá definir la especificación de un Tipo de Dato Abstracto que permita resolver un problema.
No deben programar.
Solo deben describir el comportamiento de la estructura.*



Actividad

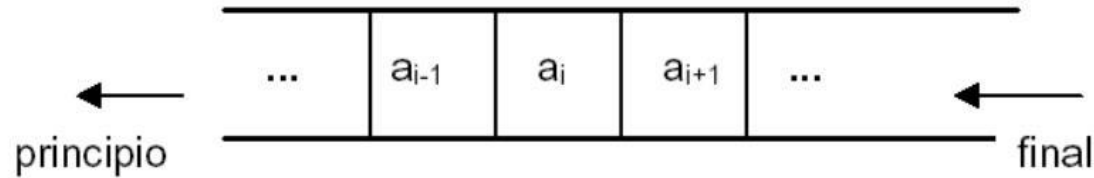
TDA COLA

Definición



Def: una cola es una lista ordenada de elementos en la que todas las inserciones se realizan por un extremo (frente o principio) y las supresiones se realizan por el otro (final).

Estructura FIFO (*First In First Out*): "primero en entrar primero en salir"



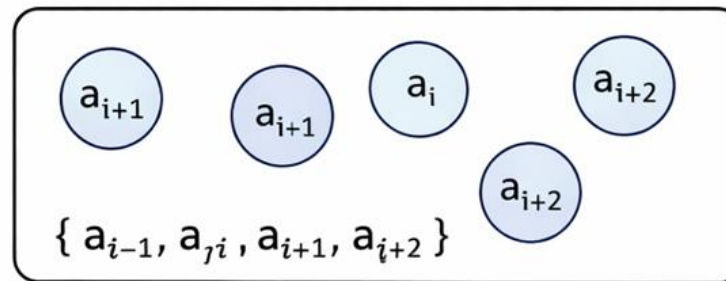
Actividad

TDA CONJUNTO

Definición

Def: un conjunto es una colección de elementos en la que *no* existe orden entre los elementos y no se permiten elementos repetidos. Los elementos pueden agregarse, eliminarse o consultarse para verificar si pertenecen al conjunto.

Estructura **SIN ORDEN (sin duplicados)**



Observación: El orden de los elementos no tiene significado.

Por ejemplo: $\{ a, b, c, d \} = \{ d, a, c, b \}$ porque en un conjunto por posición.

Actividad

TDA DICCIONARIO

Definición

Def: un diccionario es una colección de **pares clave-valor**, donde cada clave identifica de manera única un valor asociado. Permite almacenar información de forma que **a partir de una clave se pueda recuperar su valor correspondiente**.

Estructura de Diccionario

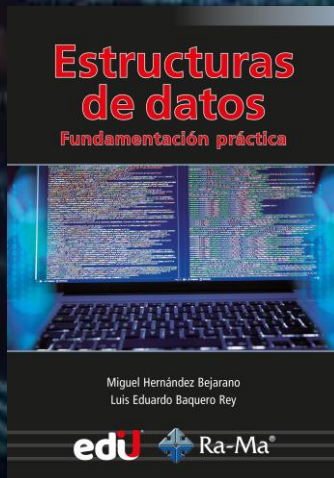
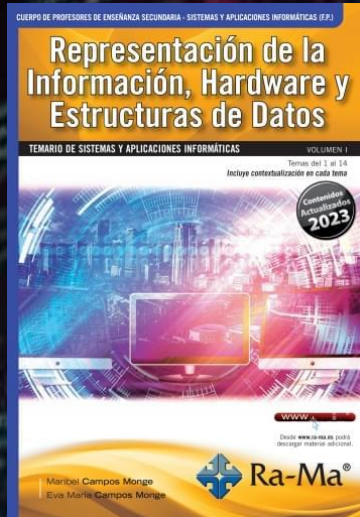
Clave	Valor
Juan	1154321000
Ana	1167890000
Pedro	1123456789

- cómo agregar
- cómo eliminar
- cómo consultar
- cómo saber si existe una clave



Observación

Cada **clave es única** y permite acceder a su valor asociado.
Diccionario["Ana"] → 1167890000



BIBLIOGRAFIA



Campos Monge, M., & Campos Monge, E. M. (2023). *Representación de la información, hardware y estructuras de datos*. RAMA.

Hernández Bejarano, M., & Baquero Rey, L. E. (2022). *Estructuras de datos: Fundamentación práctica*. RAMA.

Liskov, B., & Zilles, S. (1974). *Programming with Abstract Data Types*. ACM SIGPLAN Notices, 9(4), 50–59.

— Artículo fundacional donde se formaliza el concepto de Abstract Data Type.

Parnas, D. L. (1972). *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM, 15(12), 1053–1058.

— Introduce el principio de ocultamiento de información, base conceptual del TDA.



PROXIMO ENCUENTRO



Clase 2 – Pila y Cola: del contrato al código

TDA Pila y TDA Cola. Implementación estática con arreglos.
Idea de representación interna.